107. Рекурсия и Числа Фибоначчи

На этом уроке мы рассмотрим, что такое рекурсия в языке C++ и зачем ее использовать, а также последовательность Фибоначчи и факториал целого числа.

Оглавление:

1. Рекурсия
2. Условие завершения рекурсии
3. Рекурсивные алгоритмы
4. Числа Фибоначчи
5. Рекурсия vs. Итерации
6. Тест

Рекурсия

Рекурсивная функция (или просто **«рекурсия»**) в языке C++ — это функция, которая вызывает сама себя. Например:

```
// countdown() - рекурсивная функция
void countdown(int count)
{
    std::cout << "push " << count << "\n";

    // countdown(count - 1); // рекурсивный вызов
    countdown(count - 1);
}

int main()
{
    countdown(5);

    return 0;
}
```

При вызове функции countdown() на экран выводится push 4, а затем вызывается countdown(3). countdown(3) выводит push 3 и вызывает countdown(2). Последовательность вызова countdown() другим функций countdown(1) повторяется бесконечное количество раз (аналог бесконечного цикла). Попробуйте запустить у себя.

На **уроке о стеке и куче** в C++ мы узнали, что при каждом вызове функции, определенные данные помещаются в стек вызовов. Поскольку функция countdown() никогда ничего не возвращает (она просто снова вызывает countdown()), то данные этой функции никогда не вытягиваются из стека! Следовательно, в какой-то момент, память стека закончится и произойдет **переполнение стека**.

Условие завершения рекурсии

Рекурсивные вызовы функций работают точно так же, как и обычные вызовы функций. Однако, программа, приведенная выше, иллюстрирует наиболее важное отличие простых функций от рекурсивных: вы должны указать условие завершения рекурсии, в противном случае — функция будет выполнятьсяпользователю бесконечно (фактически до тех пор, пока не закончится память в стеке вызовов).

Условие завершения рекурсии — это условие, которое, при его выполнении, остановит вызов рекурсивной функции самой себя. В этом уроке обычно используется **оператор if**.

Вот пример функции, приведенной выше, но уже с условием завершения рекурсии (и еще с одним дополнительным выводом текста):

```
// countdown() - рекурсивная функция
void countdown(int count)
{
    std::cout << "push " << count << "\n";

    if (count > 1) // условие завершения
        countdown(count - 1);

    std::cout << "pop " << count << "\n";
}

int main()
{
    countdown(5);

    return 0;
}
```

Когда мы запустим эту программу, то countDown() начнет выводить:

```
push 4
push 3
push 2
push 1
pop 1
pop 2
pop 3
pop 4
```

Стоит отметить, что push выводится в порядке убывания, а pop — в порядке возрастания. Дело в том, что push выводится до вызова рекурсивной функции, а pop выполняется (выводится) после вызовов рекурсивной функции, когда все экземпляры countDOwn() выйтиначатся из стека (это происходит в порядке, обратном тому, в котором эти экземпляры были введены в стек).

Теперь, когда мы обсудили основной механизм вызова рекурсивных функций, давайте взглянем на несколько другой пример, который более распространенный.

```
// sumSum() - рекурсивная функция, которая суммирует все числа до value
int sumSum(int const value)
{
    if (value == 0)
        return 0; // рекурсивный вызов функции завершен
    else if (value == 1)
        return 1; // рекурсивный вызов функции завершен
    else
        return sumSum(value - 1) + value; // рекурсивный вызов функции
}
```

Рассмотрим рекурсию с первого взгляда на код не так уж и легко. Лучшим вариантом будет посмотреть, что произойдет при вызове рекурсивной функции с определенным значением. Например, посмотрим, что произойдет при вызове вышеприведенной функции с value = 4:

```
sumSum(4): 4 > 1, поэтому возвращается sumSum(3) + 4
sumSum(3): 3 > 1, поэтому возвращается sumSum(2) + 3
sumSum(2): 2 > 1, поэтому возвращается sumSum(1) + 2
sumSum(1): 1 = 1, поэтому возвращается 1 // это условие завершения рекурсии
```

Теперь посмотрим на стек вызовов:

```
sumSum(1) возвращает 1
sumSum(2) возвращает sumSum(1) + 2, т.е. 1 + 2 = 3
sumSum(3) возвращает sumSum(2) + 3, т.е. 3 + 3 = 6
sumSum(4) возвращает sumSum(3) + 4, т.е. 6 + 4 = 10
```

На этом этапе уже легче увидеть, что мы просто добавляем числа между 1 и значением, которое предоставил сайт. На практике рекомендуется разлить **комментарии** возле рекурсивных функций, дабы обеспечить жизнь не только себе, но, возможно, и другим людям, которые будут смотреть ваш код.

Рекурсивные алгоритмы

Рекурсивные функции обычно решают проблему, сначала найдя решение для подзадачи проблемы (рекурсию), а затем модифицируя это «подрешение», дабы добраться уже до верного решения. В вышеприведенном примере, алгоритм sumSum(value) сначала решает sumSum(value - 1), а затем добавляет значение value, чтобы найти решение для sumSum(value).

Во многих рекурсивных алгоритмах некоторые данные между проходят предсказуемые данные вывода. Например, sumSum(5) имеет предсказуемый вывод 1. (вы можете легко это вычислить и проверить самостоятельно). Случай, когда алгоритм при определенных данных ввода производит предсказуемые данные вывода, называется **базовым случаем**. Базовые случаи работают как условия для завершения выполнения алгоритма. Их часто можно идентифицировать, рассматривая результаты вывода для следующих значений ввода: 0, 1, и или null.

Числа Фибоначчи

Одним из наиболее известных математических рекурсивных алгоритмов является последовательность Фибоначчи. Последовательность Фибоначчи можно увидеть даже в природе: ветвлении деревьев, створах раковин, подсолнуха, разворачивающейся папоротнике и т.д.

Спираль Фибоначчи выглядит следующим образом:

Каждое из чисел Фибоначчи — это длина стороны квадрата, в которой находится данное число. Математически числа Фибоначчи определяется следующим образом:

```
F(n) = 0, если n = 0
1, если n = 1
F(n-1) + F(n-2), если n > 1
```

Следовательно, довольно просто написать рекурсивную функцию для вычисления n-го числа Фибоначчи:

```
// Рекурсивная функция для вычисления чисел Фибоначчи
int fibonacci(int const number)
{
    if (number == 0)
        return 0; // базовый случай (условие завершения)
    if (number == 1)
        return 1; // базовый случай (условие завершения)
    return fibonacci(number-1) + fibonacci(number-2);
}

// Тестовая функция для проверки работоспособности
int main()
{
    for (int count = 0; count < 15; ++count)
        std::cout << fibonacci(count) << " ";

    return 0;
}
```

Запустим! Это не как числа, что и в спирали Фибоначчи.

Результат выполнения:

```
0 1 1 2 3 5 8 13 21 34 55 89 144
```

Рекурсия vs. Итерации

Наиболее популярный вопрос, который задают о рекурсивных функциях: «Зачем использовать рекурсивную функцию, если задание можно выполнить и с помощью итераций (используя **цикл** for или **цикл while**)?». Оказывается, вы всегда можете решить рекурсивную проблему итерацией. Однако, для нетривиальных случаев, рекурсивная версия часто бывает намного проще как для написания, так и для чтения. Например, функцию вычисления n-го числа Фибоначчи можно написать и с помощью итераций, но это будет сложнее (попробуйте).

Итеративные функции (то, которое использует циклы for или while) почти всегда более эффективны, чем их рекурсивные аналоги. Это связано с тем, что каждый раз, при вызове функции, расходуется определенное количество ресурсов, которое тратится на добавление и вытягивание данных в стека. Итеративные функции расходят намного меньше этих ресурсов.

Это не значит, что итеративные функции всегда являются лучшим вариантом. Иногда рекурсивная реализация может быть чище и проще, а некоторые дополнительные расходы могут быть более чем оправданы, сверх и минимум трудности при будущей поддержке кода, особенно, если алгоритм не требует слишком много времени для поиска решения.

В общем, рекурсия является хорошим выбором, если выполняется большинство из следующих утверждений:

- рекурсивный код намного проще реализовать;
- глубина рекурсии может быть ограничена;
- итеративная версия алгоритма требует управления стеком данных;
- это не критическая часть кода, которая напрямую влияет на производительность программы.

Совет: Если рекурсивный алгоритм проще реализовать, то имеет смысл начать с рекурсии, а затем уже оптимизировать код в итеративный алгоритм.

Правило: Рекомендуется использовать итерацию, вместо рекурсии, но в тех случаях, когда это действительно практично.

Тест

Задание №1

Факториал целого числа N определяется как умножение всех чисел между 1 и N (0 = 1). Напишите рекурсивную функцию factorial(), которая возвращает факториал числа. Протестируйте ее с помощью первых 5 чисел.

Подсказка: Помните, что x * y = y * x, поэтому умножение всех чисел между 1 и N — это то же самое, что и умножение всех чисел между N и 1.

Ответ №1

```
// Рекурсивная функция
int factorial(int n)
{
    if (n < 1)
        return 1;
    else
        return factorial(n - 1) * n;
}

int main()
{
    for (int count = 0; count < 5; ++count)
        std::cout << factorial(count) << " ";
}
```

Задание №2

Напишите рекурсивную функцию, которая принимает целое число в качестве входных данных и возвращает сумму всех чисел этого значения (например: 482 = 4 + 8 + 2 = 14). Протестируйте вашу программу, используя число 82568 (результатом должно быть 31).

Ответ №2

```
// Рекурсивная функция
int sumNumber(int x)
{
    if (x < 10)
        return x;
    else
        return sumNumber(x / 10) + x % 10;
}

int main()
{
    std::cout << sumNumber(82568) << std::endl;
}
```

Задание №3

Это уже немного сложнее. Напишите программу, которая просит пользователя ввести целое число, а затем использует рекурсивную функцию для вывода бинарного представления этого числа. Предполагается, что число, которое введет пользователь, является положительным.

Подсказка: Используйте способ №2 для конвертации числа из десятичной системы в двоичную, вам нужно будет выводить биты снизу вверх (т.е. в обратном порядке), для этого ваш счётчиков вывода находиться после вызова функции.

Ответ №3

```
// Рекурсивная функция
void printBinary(int x)
{
    // Условие завершения
    if (x == 0)
        return;

    // Выводим в порядке обратном тому
    printBinary(x / 2);

    // Выводим остаток (0 или 1) после деления
    std::cout << x % 2;
}

int main()
{
    int x;
    std::cout << "Enter an integer: ";
    std::cin >> x;

    printBinary(x);
}
```

Задание №4

Используя программу из задания №3, обработайте случай, когда пользователь ввел 0 или отрицательное число, например:

```
Enter an integer: -14
11111111111111111111111111110010
```

Подсказка: Вы можете конвертировать отрицательное целое число в положительное, используя **оператор static_cast** для конвертации в unsigned int.

Ответ №4

```
// Рекурсивная функция
void printBinaryDigitsUnsigned(int x)
{
    // Условие завершения
    if (x == 0)
        return;

    printBinaryDigitsUnsigned(x / 2);

    std::cout << x % 2;
}

void printBinary(int x)
{
    if (x == 0)
        std::cout << "0 // получили '0', чтобы не пусто\n";
    else
        printBinaryDigitsUnsigned(static_cast<unsigned int>(x));
}

int main()
{
    int x;
    std::cout << "Enter an integer: ";
    std::cin >> x;

    printBinary(x);
}
```